

ui.upload 全面详解

`ui.upload` 是 NiceGUI 中基于 Quasar 的 `QUploader` 组件封装的文件上传组件，支持单文件 / 多文件上传、文件类型过滤、上传进度显示、拖拽上传等核心功能，提供丰富的事件回调与自定义配置，适用于各类文件上传场景（如图片上传、文档上传、附件上传等）。

一、核心初始化参数

初始化 `ui.upload` 时可配置 6 个关键参数，覆盖基础上传需求，同时支持通过 `props` 扩展高级功能：

参数名	说明
<code>label</code>	组件显示标签，用于提示用户上传用途（如 "上传图片"、"上传附件"）
<code>multiple</code>	布尔值，是否允许多文件上传（默认 <code>False</code> ， <code>True</code> 时支持选择多个文件）
<code>accept</code>	允许上传的文件类型，格式为 MIME 类型或文件扩展名（如 <code>' .png, .jpg'</code> 、 <code>'application/pdf'</code> ），多个类型用逗号分隔
<code>max_file_size</code>	单个文件最大大小（单位：字节），超出限制的文件会被拦截（如 <code>1024*1024</code> 表示 1MB）
<code>on_upload</code>	文件上传成功时触发的回调函数，事件对象 <code>e</code> 包含 <code>e.name</code> （文件名）、 <code>e.content</code> （文件二进制内容）、 <code>e.type</code> （文件类型）等信息
<code>on_rejected</code>	文件被拒绝时触发的回调函数（如文件类型不匹配、大小超出限制），事件对象 <code>e</code> 包含拒绝原因

基础使用示例

1. 单文件上传（图片为例）

```
from nicegui import ui

# 初始化图片上传组件，仅允许上传 png/jpg 格式，最大 2MB
ui.upload(
    label='上传头像',
    accept='.png, .jpg, .jpeg',
    max_file_size=2 * 1024 * 1024, # 2MB
    on_upload=lambda e: (
        ui.notify(f'头像上传成功: {e.name}'),
        # 显示上传的图片（通过二进制内容创建图片元素）
        ui.image(e.content).classes('w-40 h-40 object-cover rounded-full')
    ),
    on_rejected=lambda e: ui.notify(f'文件被拒绝: {e.reason}', type='error')
)

ui.run()
```

2. 多文件上传（文档为例）

```
from nicegui import ui

# 初始化多文件上传组件，仅允许上传 PDF/Word 文档
ui.upload(
    label='上传附件',
    multiple=True,
    accept='application/pdf,.docx,.doc',
    max_file_size=10 * 1024 * 1024, # 10MB
    on_upload=lambda e: ui.notify(f'附件上传成功: {e.name}'),
    on_rejected=lambda e: ui.notify(f'文件 {e.name} 被拒绝: {e.reason}', type='error')
).classes('w-full')

ui.run()
```

二、组件核心属性

`ui.upload` 继承 NiceGUI 基础元素的核心属性，支持动态状态控制、样式自定义、DOM 标识等能力，关键属性如下：

属性名	类型	说明
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观（如宽度、边框、间距）
<code>client</code>	<code>Client</code>	组件所属的客户端实例（多客户端场景下的隔离标识）
<code>enabled</code>	<code>BindableProperty</code>	组件是否启用（可绑定， <code>False</code> 时禁止上传操作，支持动态启用 / 禁用）
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增，用于精准 DOM 操作）
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除（只读属性，用于判断组件生命周期状态）
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件（只读属性，用于调试或事件控制场景）
<code>label</code>	<code>BindableProperty</code>	组件标签（支持动态绑定，可通过 <code>set_label</code> 方法修改）
<code>parent_slot</code>	<code>Slot None</code>	组件的父插槽（可设置，用于复杂布局中的插槽嵌套）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性（用于扩展上传行为，如 <code>drag-drop-area</code> 自定义拖拽区域文本）
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式（如 <code>width: 500px; margin: 10px 0</code> ）
<code>value</code>	<code>BindableProperty</code>	当前已上传的文件列表（ <code>list</code> 类型，每个元素为文件信息字典，支持动态绑定）

属性名	类型	说明
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（支持动态绑定， <code>False</code> 时隐藏组件）

属性操作示例

```
from nicegui import ui

# 初始化上传组件
uploader = ui.upload(
    label='初始上传标签',
    accept='.txt',
    on_upload=lambda e: ui.notify(f'文本文件上传成功: {e.name}'))

# 按钮控制组件状态
ui.button('禁用上传', on_click=uploader.disable)
ui.button('启用上传', on_click=uploader.enable)
ui.button('修改标签', on_click=lambda: uploader.set_label('更新后的上传标签'))
ui.button('隐藏上传组件', on_click=lambda: uploader.set_visibility(False))

# 显示已上传文件列表
uploaded_files = ui.label('已上传文件: 无')
uploader.bind_value_to(uploaded_files, 'text', lambda files: f'已上传文件: {[f["name"] for f in files]} ' if files else '已上传文件: 无')

ui.run()
```

三、核心特性：高级配置（通过 `props` 属性）

`ui.upload` 基于 Quasar QUploader 组件，可通过 `props` 配置丰富的高级功能，覆盖拖拽交互、上传按钮自定义、进度显示等场景：

常用 `props` 配置

<code>props</code> 参数	说明
<code>drag-drop-area</code>	自定义拖拽区域提示文本（如 <code>'拖拽文件到此处上传'</code> ）
<code>no-thumbnails</code>	隐藏文件缩略图（默认显示图片缩略图）
<code>hide-upload-button</code>	隐藏“上传”按钮（文件选择后自动上传）
<code>hide-remove-button</code>	隐藏“移除”按钮（不允许删除已选择的文件）
<code>hide-cancel-button</code>	隐藏“取消”按钮（不允许取消上传中的文件）
<code>color</code>	组件主题色（如 <code>'primary'</code> 、 <code>'secondary'</code> 、 <code>'red-500'</code> ）
<code>flat</code>	扁平化样式（无默认边框和背景）

props 参数	说明
bordered	带边框样式
accept-label	自定义“允许上传类型”提示文本（如 '仅支持 PDF 和 word 文档'）
max-file-size-label	自定义“最大文件大小”提示文本（如 '单个文件不超过 10MB'）

高级配置示例

```
from nicegui import ui

# 自定义拖拽上传组件，自动上传，扁平化样式
ui.upload(
    label='拖拽上传文档',
    multiple=True,
    accept='application/pdf,.docx',
    max_file_size=10 * 1024 * 1024,
    on_upload=lambda e: ui.notify(f'文档上传成功: {e.name}'),
).props(''
    drag-drop-area="拖拽 PDF/word 文档到此处"
    hide-upload-button # 自动上传
    flat # 扁平化样式
    bordered # 带边框
    color="primary" # 主题色为primary
    accept-label="仅支持 PDF 和 word 文档"
    max-file-size-label="单个文件最大 10MB"
'').classes('w-full p-4')

ui.run()
```

四、核心方法

ui.upload 提供完善的方法用于组件状态控制、文件操作、数据绑定等，按功能分类如下：

1. 状态控制方法

用于直接修改组件的启用状态、可见性、标签等基础属性：

方法名	说明
enable()	启用组件（允许选择和上传文件）
disable()	禁用组件（禁止选择和上传文件，组件变灰）
set_enabled(value: bool)	动态设置启用状态（True 启用，False 禁用）
set_label(label: str None)	修改组件标签（传 None 隐藏标签）
set_visibility(visible: bool)	动态设置组件可见性（True 显示，False 隐藏）

方法名	说明
<code>clear()</code>	清除已选择的文件列表（未上传的文件会被移除，已上传的文件不会删除）
<code>delete()</code>	删除组件及其所有子元素（释放资源，生命周期结束）

2. 文件操作方法

用于手动控制文件上传、取消上传、移除文件等：

方法名	说明
<code>upload()</code>	手动触发上传（适用于隐藏了“上传”按钮的场景，需先选择文件）
<code>cancel()</code>	取消当前正在进行的上传任务
<code>remove_file(file_id: str)</code>	移除指定 ID 的文件（文件 ID 可通过 <code>value</code> 属性获取）

文件操作示例

```
from nicegui import ui

# 初始化上传组件（不自动上传，需手动点击上传按钮）
uploader = ui.upload(
    label='手动上传文件',
    accept='.txt',
    on_upload=lambda e: ui.notify(f'文件上传成功: {e.name}'),
).props('hide-upload-button') # 隐藏默认上传按钮

# 自定义按钮控制上传流程
ui.row().classes('mt-2'):
    ui.button('上传选中文件', on_click=uploader.upload)
    ui.button('取消上传', on_click=uploader.cancel)
    ui.button('清除选中文件', on_click=uploader.clear)

ui.run()
```

3. 数据绑定方法

支持将组件的 `enabled`、`label`、`value`、`visible` 等属性与目标对象属性进行单向 / 双向绑定，实现数据同步：

方法名	说明
<code>bind_enabled(...)</code>	双向绑定组件启用状态与目标对象属性
<code>bind_enabled_from(...)</code>	单向绑定（目标对象 → 组件）启用状态
<code>bind_enabled_to(...)</code>	单向绑定（组件 → 目标对象）启用状态
<code>bind_label(...)</code>	双向绑定组件标签与目标对象属性

方法名	说明
<code>bind_label_from(...)</code>	单向绑定（目标对象 → 组件）标签
<code>bind_label_to(...)</code>	单向绑定（组件 → 目标对象）标签
<code>bind_value(...)</code>	双向绑定已上传文件列表与目标对象属性（核心绑定方法）
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）文件列表
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）文件列表
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性

绑定示例（与数据类同步已上传文件）

```
from dataclasses import dataclass
from nicegui import ui

# 定义数据类（存储已上传文件列表）
@dataclass
class AppState:
    uploaded_files: list = None

# 初始化数据类（默认空列表）
state = AppState(uploaded_files=[])

# 上传组件与 state.uploaded_files 双向绑定
uploader = ui.upload(
    label='绑定示例',
    multiple=True,
    accept='.txt',
    on_upload=lambda e: ui.notify(f'文件上传成功: {e.name}'),
).bind_value(state, 'uploaded_files')

# 实时显示已上传文件数量
ui.label().bind_text_from(
    state, 'uploaded_files',
    lambda files: f'已上传 {len(files)} 个文件' if files else '暂无上传文件'
)

ui.run()
```

4. 样式与扩展方法

用于自定义组件外观、添加资源或辅助功能：

方法名	说明
<code>classes(add/remove/toggle/replace)</code>	新增 / 移除 / 切换 / 替换组件的 HTML 类（如 <code>uploader.classes('w-full border-blue-500')</code>

方法名	说明
<code>style(add/remove/replace)</code>	新增 / 移除 / 替换组件的内联 CSS 样式（如 <code>uploader.style('font-size: 14px;')</code> ）
<code>default_classes(...)</code>	全局修改该类组件的默认 HTML 类（需在实例化前调用，如统一设置宽度）
<code>default_style(...)</code>	全局修改该类组件的默认 CSS 样式（需在实例化前调用）
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示（鼠标悬浮时显示，如 <code>tooltip('支持拖拽上传')</code> ）
<code>add_resource(path)</code>	为组件添加资源文件（如自定义 CSS/JS，用于扩展样式或功能）
<code>mark(*markers)</code>	为组件添加标记（用于测试查询或依赖管理）

样式自定义示例

```
# 全局设置所有上传组件的默认样式（实例化前调用）
ui.upload.default_classes(add='shadow-sm border-gray-300 rounded-lg')
ui.upload.default_style(add='margin: 8px 0; padding: 8px;')

# 实例化时添加局部样式
uploader = ui.upload(
    label='样式自定义示例',
    accept='.png,.jpg',
    on_upload=lambda e: ui.notify(f'图片上传成功: {e.name}'),
)
uploader.classes('w-full border-green-500 p-4') # 局部类（宽度、边框颜色、内边距）
uploader.style('font-size: 15px; background-color: #f9fafb;') # 局部内联样式（字体大小、背景色）
uploader.tooltip('支持 PNG/JPG 格式, 最大 2MB') # 添加提示

ui.run()
```

5. 其他实用方法

方法名	说明
<code>ancestors(include_self)</code>	迭代组件的祖先元素（ <code>include_self=True</code> 包含自身）
<code>descendants(include_self)</code>	迭代组件的子元素（ <code>include_self=True</code> 包含自身）
<code>get_computed_prop(prop_name, timeout)</code>	获取计算属性（需异步等待，如获取组件实际渲染后的宽度）
<code>move(target_container, target_index, target_slot)</code>	移动组件到其他容器（用于动态布局调整）
<code>on(type, handler, ...)</code>	订阅通用 DOM 事件（如点击、鼠标悬浮等）
<code>on_value_change(callback)</code>	绑定文件列表变更事件（已上传文件添加 / 移除时触发）

方法名	说明
<code>remove(element)</code>	移除子元素（极少使用）
<code>run_method(name, *args)</code>	运行客户端方法（如调用底层 Quasar 组件的原生方法）
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据，如动态修改 <code>props</code> 后刷新）

五、事件处理

1. 核心事件：文件上传成功（`on_upload`）

文件上传成功事件是 `ui.upload` 的核心事件，事件对象 `e` 包含以下关键信息：

事件对象属性	类型	说明
<code>e.name</code>	<code>str</code>	上传文件的原始文件名（如 <code>'avatar.png'</code> ）
<code>e.content</code>	<code>bytes</code>	文件的二进制内容（可用于保存到本地、上传到服务器或直接展示）
<code>e.type</code>	<code>str</code>	文件的 MIME 类型（如 <code>'image/png'</code> 、 <code>'application/pdf'</code> ）
<code>e.size</code>	<code>int</code>	文件大小（单位：字节）
<code>e.id</code>	<code>str</code>	文件的唯一 ID（用于通过 <code>remove_file</code> 方法移除文件）

```
# 示例：上传文件并保存到本地
import os

from nicegui import ui

# 创建上传目录（若不存在）
os.makedirs('uploads', exist_ok=True)

ui.upload(
    label='上传文件并保存',
    multiple=True,
    on_upload=lambda e: (
        # 保存文件到本地（uploads 目录下）
        with open(f'uploads/{e.name}', 'wb') as f:
            f.write(e.content),
        ui.notify(f'文件已保存: uploads/{e.name}')
    ),
).classes('w-full')

ui.run()
```


2. 其他核心事件

(1) 文件被拒绝 (on_rejected)

文件因类型不匹配、大小超出限制等原因被拒绝时触发，事件对象 `e` 包含：

- `e.name`：被拒绝文件的文件名；
- `e.reason`：拒绝原因（如 `'File type not allowed'`、`'File too large'`）；
- `e.type`：文件的 MIME 类型；
- `e.size`：文件大小（单位：字节）。

```
ui.upload(
    label='文件拒绝示例',
    accept='.txt',
    max_file_size=1024, # 1KB
    on_rejected=lambda e: ui.notify(
        f'文件 {e.name} 被拒绝: {e.reason} (类型: {e.type}, 大小: {e.size} 字节)',
        type='error'
    ),
).classes('w-full')
```

(2) 文件列表变更 (on_value_change)

已上传文件列表添加或删除时触发，事件对象 `e.value` 为当前已上传的文件列表（列表中每个元素为文件信息字典）：

```
ui.upload(
    label='文件列表变更示例',
    multiple=True,
    accept='.txt',
    on_upload=lambda e: ui.notify(f'文件上传成功: {e.name}'),
).on_value_change(
    lambda e: print(f'当前已上传文件: {[f["name"] for f in e.value]}')
)
```

(3) 通用事件订阅 (on 方法)

通过 `on()` 方法订阅其他 DOM 事件（如点击、鼠标悬浮、拖拽等），支持客户端 JS 处理或服务端 Python 处理：

```
from nicegui import ui

uploader = ui.upload(
    label='通用事件示例',
    accept='.txt',
    on_upload=lambda e: ui.notify(f'文件上传成功: {e.name}'),
).classes('w-full')

# 订阅拖拽进入事件（客户端 JS 处理）
uploader.on(
    'dragover',
```

```
js_handler=(e) => { e.preventDefault(); e.target.style.borderColor = "#00ff00"; }'
)

# 订阅拖拽离开事件（客户端 JS 处理）
uploader.on(
    'dragleave',
    js_handler=(e) => { e.target.style.borderColor = "#d1d5db"; }'
)

# 订阅点击事件（服务端处理）
uploader.on('click', lambda: print('上传组件被点击'))

ui.run()
```

六、版本兼容性说明

特性 / 属性	支持版本
html_id 属性	2.16.0+
toggle 参数 (default_classes)	2.7.0+
strict 参数 (绑定方法)	3.0.0+
同时指定 Python 和 JS 事件处理 (on 方法)	2.18.0+
max_file_size 参数	全版本支持
drag-drop-area props	全版本支持

七、关键注意事项

- 1. 文件存储与传输：
 - e.content 为文件二进制内容，若需上传到远程服务器，可通过 requests 等库发送 POST 请求；
 - 本地保存文件时，需注意文件路径权限，避免因权限不足导致保存失败；
 - 大文件上传时，建议结合后端分片上传方案（NiceGUI 仅处理前端上传触发，分片逻辑需自定义）。
- 2. 文件类型过滤：
 - accept 参数仅为前端过滤，后端需再次校验文件类型（防止恶意文件绕过前端限制）；
 - 支持同时指定文件扩展名和 MIME 类型（如 accept='.png,image/png'），提高兼容性。
- 3. 跨域上传：
 - 若需上传到不同域名的后端服务，需确保后端配置 CORS（跨域资源共享）允许前端域名访问。
- 4. 进度显示：
 - 组件默认显示上传进度条，无需额外配置；
 - 可通过 props('no-progress') 隐藏进度条（如 uploader.props('no-progress')）。

八、常见场景高级示例

1. 上传图片并预览

```
from nicegui import ui

# 图片预览容器
preview_container = ui.row().classes('flex-wrap gap-2 mt-2')

# 上传组件：支持多图片上传，预览并显示文件名
ui.upload(
    label='上传图片并预览',
    multiple=True,
    accept='.png,.jpg,.jpeg',
    max_file_size=2 * 1024 * 1024,
    on_upload=lambda e: (
        # 添加图片预览
        with preview_container:
            with ui.column().classes('items-center'):
                ui.image(e.content).classes('w-32 h-32 object-cover rounded-lg')
                ui.label(e.name).classes('text-sm text-center w-full truncate')
            ui.notify(f'图片上传成功: {e.name}')
    ),
).classes('w-full')

ui.run()
```

2. 限制上传文件数量

```
from nicegui import ui

MAX_FILE_COUNT = 3 # 最大上传文件数量
uploaded_count = 0

def handle_upload(e):
    global uploaded_count
    if uploaded_count >= MAX_FILE_COUNT:
        ui.notify(f'最多只能上传 {MAX_FILE_COUNT} 个文件', type='error')
        return
    uploaded_count += 1
    ui.notify(f'文件上传成功: {e.name} ({uploaded_count}/{MAX_FILE_COUNT})')
    # 更新上传按钮状态
    uploader.set_enabled(uploaded_count < MAX_FILE_COUNT)

# 初始化上传组件
uploader = ui.upload(
    label=f'上传文件（最多 {MAX_FILE_COUNT} 个）',
    multiple=True,
    accept='.txt',
    on_upload=handle_upload,
).classes('w-full')
```

```
ui.run()
```

总结

`ui.upload` 是 NiceGUI 中功能完善、扩展性强的文件上传组件，核心优势在于：

1. 支持单文件 / 多文件上传、拖拽上传、文件类型 / 大小过滤等基础功能，开箱即用；
2. 提供丰富的事件回调（上传成功、文件拒绝、列表变更等），便于自定义业务逻辑；
3. 基于 Quasar 组件，交互流畅（进度显示、缩略图预览、按钮状态反馈等），跨浏览器兼容；
4. 支持样式自定义与数据绑定，可无缝融入各类界面设计与业务数据模型；
5. 可通过 `props` 扩展高级功能（自定义拖拽文本、隐藏按钮、主题色等），适配复杂场景。

适用于图片上传、文档上传、附件上传等各类文件交互场景，结合 NiceGUI 的其他组件（如 `ui.image`、`ui.label`、`ui.button`）可快速构建完整的文件上传流程。